VLIW Processors

CS 2410

Wonsun Ahn

University of Pittsburgh

Limits on Deep Pipelining

- Ideally, $\text{CycleTime}_{\text{Pipelined}} = \text{CycleTime}_{\text{SingleCycle}} / \text{Number of Stages}$
  - In theory, can indefinitely improve performance with more stages

- Limitation 1: **Cycle time** does not improve indefinitely
  - Latch delay + unbalanced stages (manufacturing variability)
- Limitation 2: **CPI** tends to increase with deep pipelines
  - Penalty due to branch misprediction increases
  - Stalls due to data hazards cause more bubbles

- Is there another way to improve performance?

What if we improve CPI?

- Remember the three components of performance?

$$\frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

- Pipelining focused on seconds / cycle, or cycle time

- Can we improve cycles / instruction, or CPI?
  - But the best we can get is CPI = 1, right?
  - How can an instruction be executed in less than a cycle?

Wide Issue Processors

From CPI to IPC

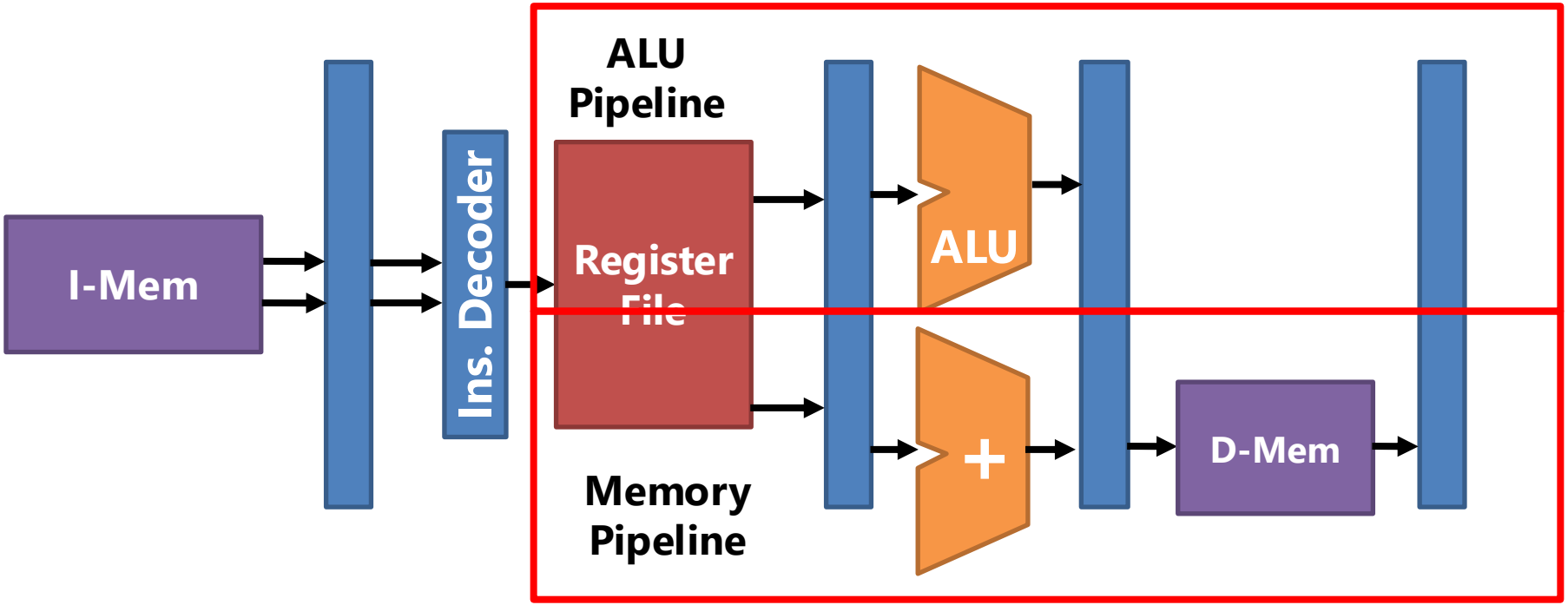
- How about if we execute **two** instructions each cycle?
  - Maybe, fetch one ALU instruction and one load/store instruction

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

- Then, **IPC (Instructions per Cycle)** = 2
  - And by extension, CPI = 1 / IPC = 0.5 !
- ***Wide-issue*** processors can execute multiple instructions per cycle

Can be achieved with mere addition of one + unit!

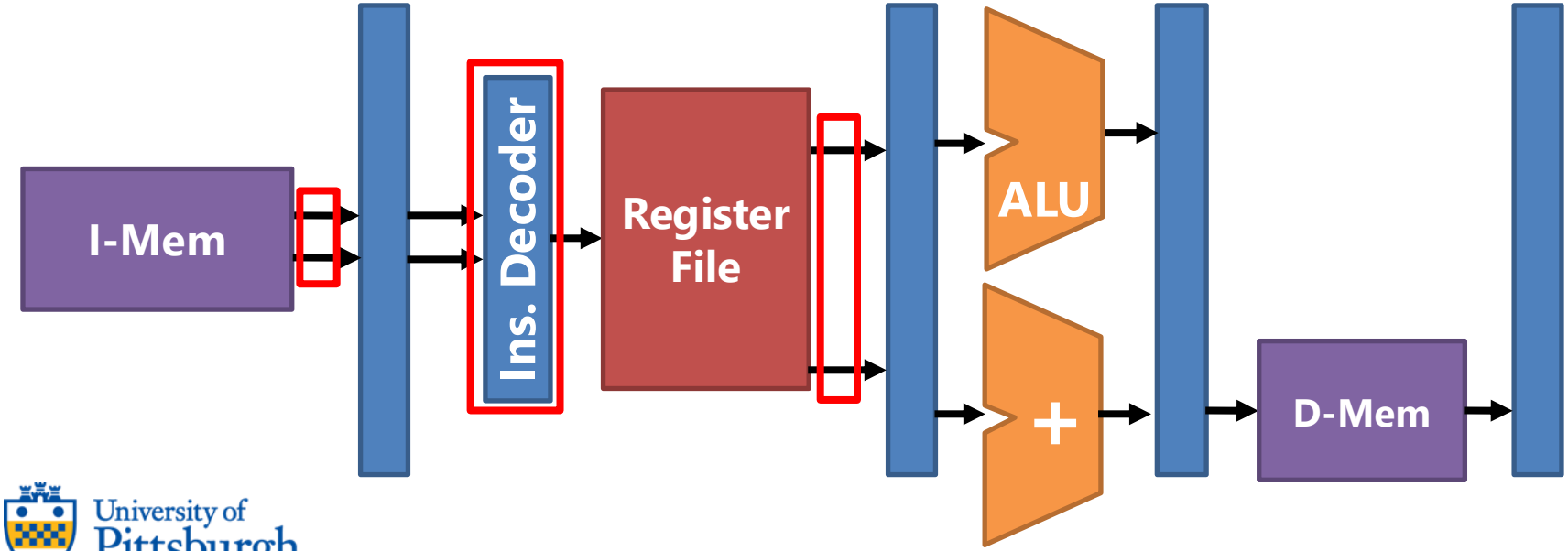
- One pipeline for ALU/Branches and one for loads and stores



- Causes contention on shared resources and structural hazards!

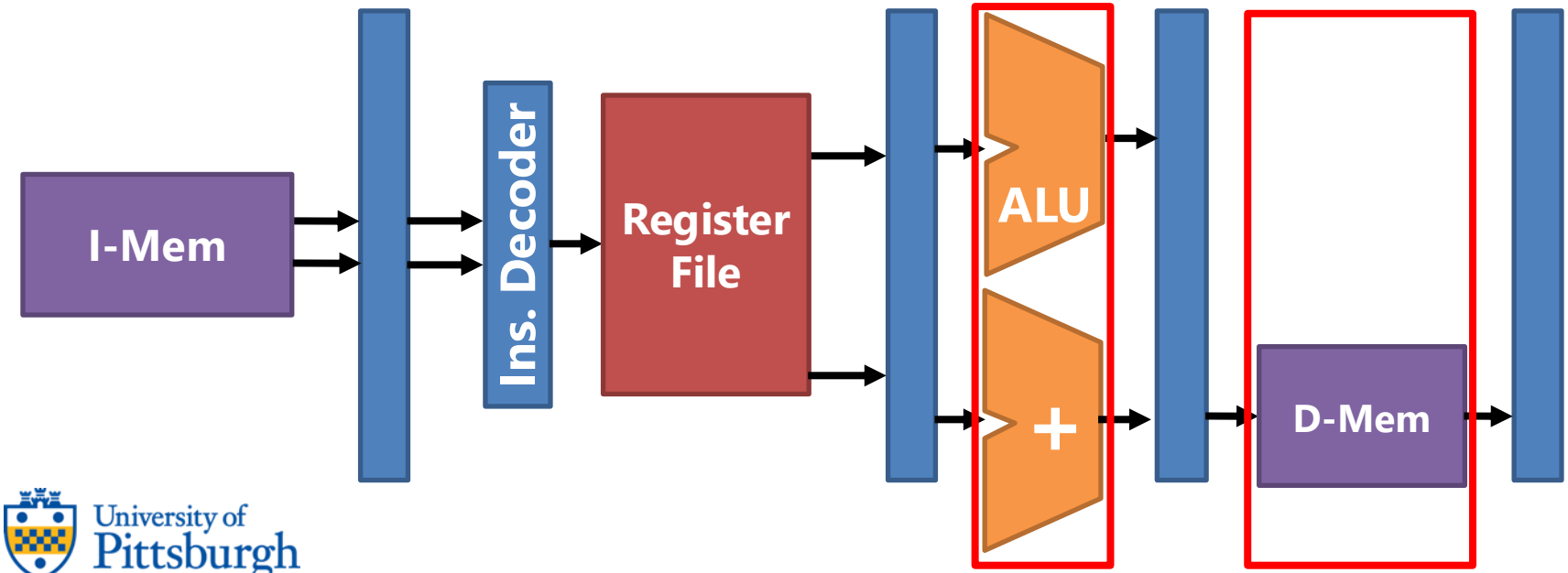
… and a few extra investments in hardware resouces

- At max, every clock cycle in this 2-wide processor:
  - 2 instructions fetched from I-MEM → 1 extra read port needed
  - 2 instructions decoded → 2 instruction decoders needed
  - 3 reads from register file → 1 extra read port needed
  - 2 writes to register file → 1 extra write port needed



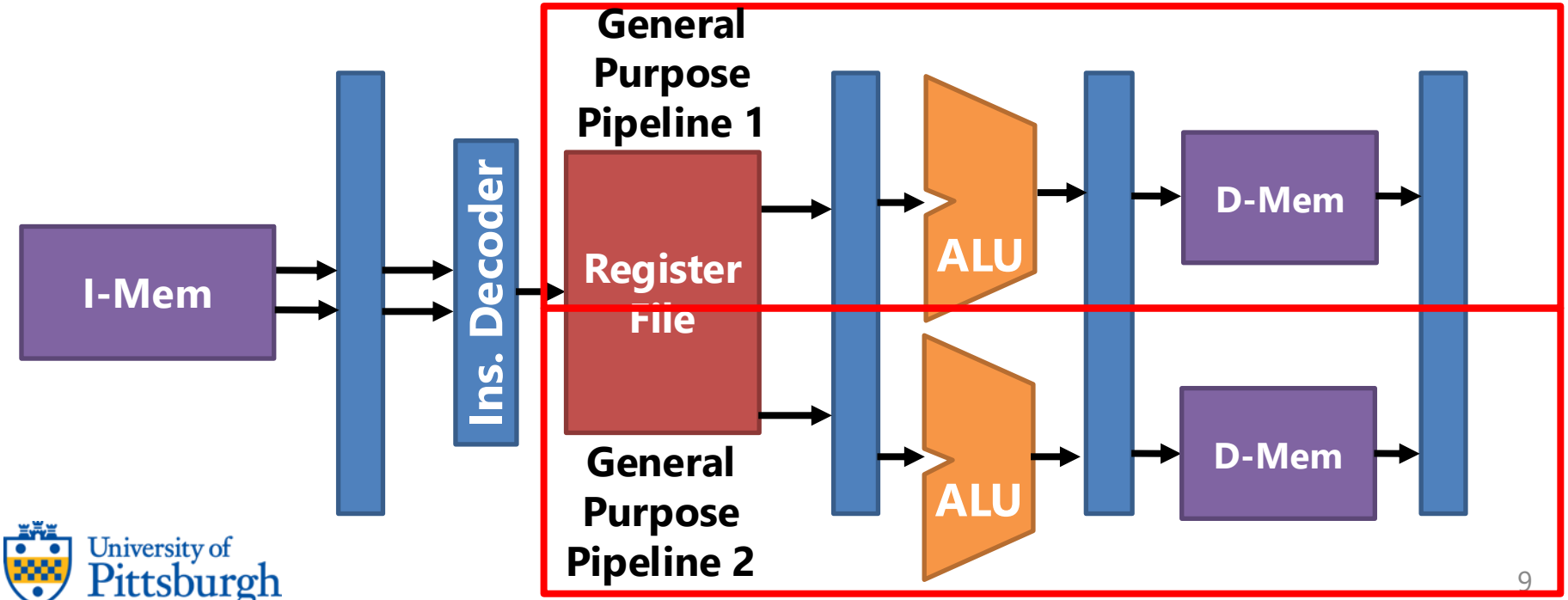
But fundamental structural hazards still remain

- Structural hazard on EX units
  - Top ALU can handle all arithmetic (+, -, *, /)
  - Bottom ALU can only handle +, needed for address calculation
- Structural hazard on MEM unit
  - ALU pipeline does not have a MEM unit to access memory



Why not just duplicate all resources?

- But this leads to **low utilization**
  - ALU/Branch type instructions will not use the MEM unit
  - Load/Store instructions will not need the full ALU
- Reality: CPUs have specialized pipelines for different instructions
  - Integer ALU pipeline, FP ALU pipeline, Load/Store pipeline, …



Example: Structural Hazard in Functional Units

- Code on the left will result in a timeline on the right
  - If it were not for the bubbles, we could have finished in 4 cycles!

```
lw   $t0,  0($s1)
lw   $t1, -4($s1)
addi $t2, $t2, -8
add  $t3, $t0, $s1
add  $t4, $s1, $s1
sw   $t5, 8($t3)
sw   $t6, 4($s1)
```

CC	ALU Pipeline	Mem Pipeline
1		lw t0
2	addi t2	lw t1
3	add t3	
4	add t4	sw t5
5		sw t6

Example: Structural Hazard in Functional Units

- With two general purpose pipelines, we could be done in 4 cycles!

```
lw   $t0,  0($s1)
lw   $t1, -4($s1)
addi $t2, $t2, -8
add  $t3, $t0, $s1
add  $t4, $s1, $s1
sw   $t5, 8($t3)
sw   $t6, 4($s1)
```

CC	Pipeline 1	Pipeline 2
1	lw t0	lw t1
2	addi t2	add t3
3	add t4	sw t5
4	sw t6	
5		

Example still finishes in 4 cycles with Reordering!

1. Create a data dependence graph for the instructions.
2. Reorder instructions while adhering to data dependencies.

```
lw   $t0,  0($s1)
lw   $t1, -4($s1)
addi $t2, $t2, -8
add  $t3, $t0, $s1
add  $t4, $s1, $s1
sw   $t5, 8($t3)
sw   $t6, 4($s1)
```

CC	ALU Pipeline	Mem Pipeline
1		lw t0
2	addi t2	lw t1
3	add t3	
4	add t4	sw t5
5		sw t6

VLIW vs. Superscalar

- There are two types of wide-issue processors

- If the compiler does **static scheduling**, the processor is called:
  - ***VLIW (Very Long Instruction Word)*** processor
  - This is what we will learn this chapter

- If the processor does **dynamic scheduling**, the processor is called:
  - ***Superscalar*** processor
  - This is what we will learn next chapter

VLIW Processors

VLIW Processor Overview

- What does **Very Long Instruction Word** mean anyway?
  - It means one instruction is *very* long!
  - Why? Because it contains multiple operations in one instruction

- A (64 bits long) VLIW instruction for our example architecture:

ALU/Branch Operation (32 bits)	Load/Store Operation (32 bits)

- An example instruction could be:

`addi $t2, $t0, -8`	`lw $t1, -4($s1)`

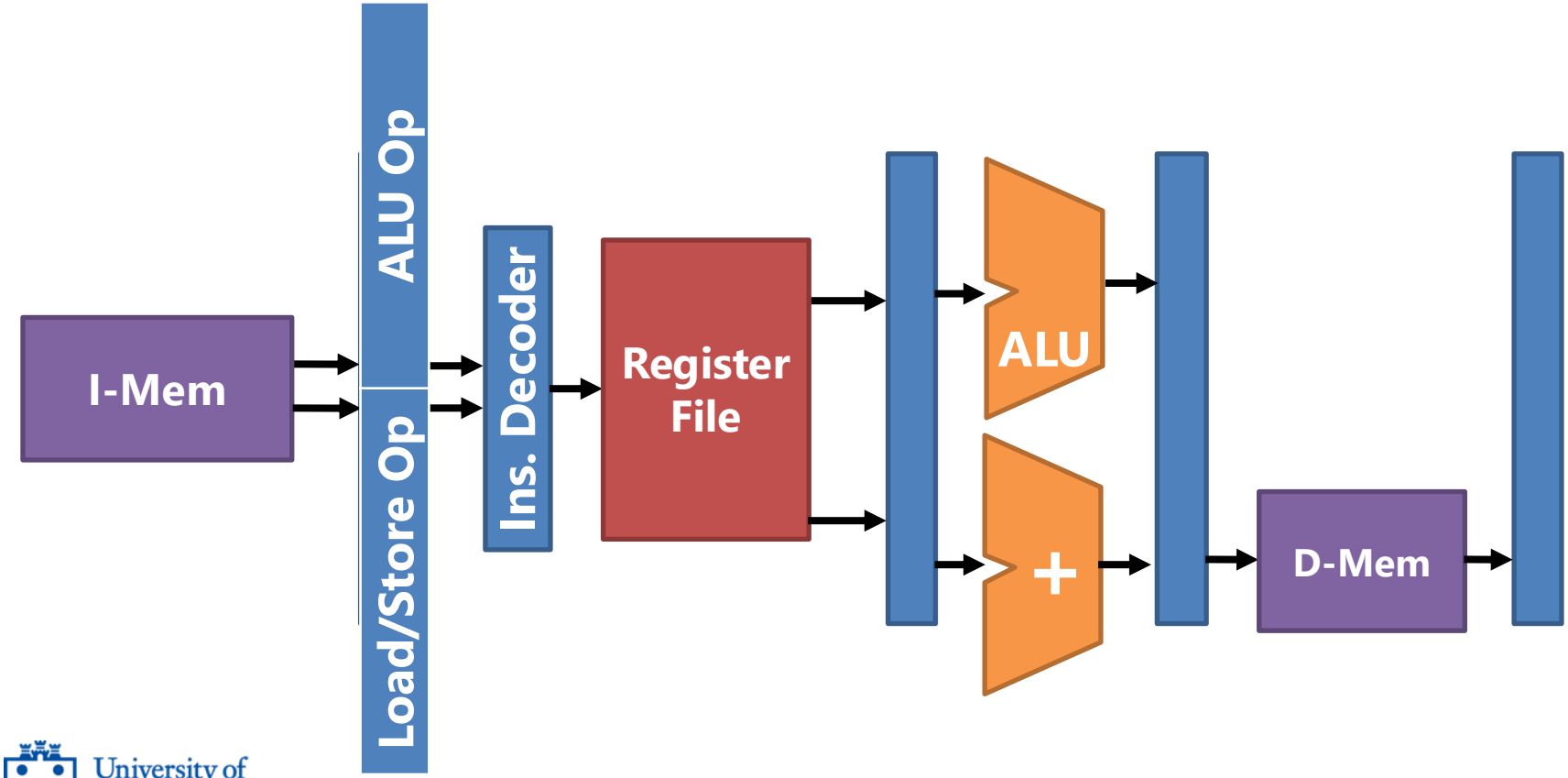
- Or another example could be:

`nop`	`lw $t1, -4($s1)`

University of Pittsburgh

A VLIW instruction is one instruction

- For all purposes, a VLIW instruction acts like one instruction
  - It moves as a unit through the pipeline



VLIW instruction encoding for example

```
nop
lw    $t0,  0($s1)
addi  $t2, $t2, -8
lw    $t1, -4($s1)
add   $t3, $t0, $s1
nop
add   $t4, $s1, $s1
sw    $t5, 8($t3)
nop
sw    $t6, 4($s1)
```

Inst	ALU Op	Load/Store Op
1	nop	lw t0
2	addi t2	lw t1
3	add t3	nop
4	add t4	sw t5
5	nop	sw t6

- Each square is an instruction. (There are 5 instructions.)
- Nops are inserted by the compiler.

VLIW instruction encoding (after reordering)

```
add   $t4, $s1, $s1
lw    $t0,  0($s1)
addi  $t2, $t2, -8
lw    $t1, -4($s1)
add   $t3, $t0, $s1
sw    $t6, 4($s1)
nop
sw    $t5, 8($t3)
```

Inst	ALU Op	Load/Store Op
1	add t4	lw t0
2	addi t2	lw t1
3	add t3	sw t6
4	nop	sw t5

- Same program with 4 instructions!

VLIW Architectures are (Very) Power Efficient

- All scheduling is done by the compiler offline
- No need for the Hazard Detection Unit
  - Nops are inserted by the compiler when necessary
- No need for a dynamic scheduler
  - Which can be even more power hungry than the HDU
- Even no need for the Forwarding Unit
  - If compiler is good enough and fill all bubbles with instructions
  - Or, may have cheap compiler-controlled forwarding in ISA

Challenges of VLIW

- All the challenges of static scheduling apply here **X 2**

- Review: what were the limitations?

  - Compiler must make **assumptions about the pipeline**
    → ISA now becomes much more than instruction set + registers
    → ISA restricts modification of pipeline in future generations

  - Compiler must do **scheduling without runtime information**
    → Length of MEM stage is hard to predict (due to Memory Wall)
    → Data dependencies are hard (must do pointer analysis)

- These limitations are exacerbated with VLIW

Not Portable due to Assumptions About Pipeline

- VLIW ties ISA to a particular processor design
  - One that is 2-wide and has an ALU op and a Load/Store op
  - What if future processors are wider or contain different ops?

- Code must be recompiled repeatedly for future processors
  - Not suitable for releasing general purpose software
  - Reason VLIW is most often used for embedded software (Because embedded software is not expected to be portable)

- Is there any way to get around this problem?

Making VLIW Software Portable

- There are mainly two ways VLIW software can become portable

1. Allow CPU to exploit parallelism according to capability
   - Analogy: multithreaded software does not specify number of cores
     - SW: Makes parallelism explicit by coding using threads
     - CPU: Exploits parallelism to the extent it has number of cores
   - Portable VLIW: ISA does not specify number of ops in instruction
     - SW: Makes parallelism explicit by using bundles
       - Bundle: a group of ops that can execute together
       - Wider processors fetch several bundles to form one instruction
       - A "stop bit" tells processor to stop fetching the next bundle
     - Intel Itanium EPIC(Explicitly Parallel Instruction Computing)
       - A general-purpose ISA that uses bundles

Making VLIW Software Portable

- There are mainly two ways VLIW software can become portable

2. Binary translation
   - Have firmware translate binary to new VLIW ISA on the fly
   - Doesn't this go against the power efficiency of VLIWs?
     - Yes, but if SW runs for long time, one-time translation is nothing
     - Translation can be cached in file system for next run
   - Transmeta processors converted x86 to an ultra low-power VLIW

- Other examples of binary translation
  - Apple Rosetta converts x86 to ARM ISA for M1 or M2 chips
  - NVIDIA GPUs convert PTX (Parallel Thread Execution) to SASS
    - SASS (Stream ASSembly) is different for every generation of GPU

Scheduling without Runtime Information

- Up to the compiler to create schedule with minimal nops
  - Use reordering to fill nops with useful operations

- All the challenges of static scheduling remain
  - Length of MEM stage is hard to predict (due to Memory Wall)
  - Data dependencies are hard to figure out (due to pointer analysis)

- And these challenges become especially acute for VLIW
  - For 4-wide VLIW, need to find 4 operations to fill "one" bubble!
  - Operations in one instruction must be data independent
    - Data forwarding will not work within one instruction (Obviously because they are executing on the same cycle)
  - Operations must also be control independent

Control Dependency Removal: Predicates

Predicates Help in Compiler Scheduling

- Predicates can enlarge "instruction window" for scheduling
  - Reordering cannot happen across control dependencies

```
lw $t1, 0($s0)
addi $t1, $t1, 1
bne $s1, $s2, else
```

Can only reorder within this block

```
then:
  lw $t2, 1($s0)
```

```
else:
  lw $t2, 2($s0)
```

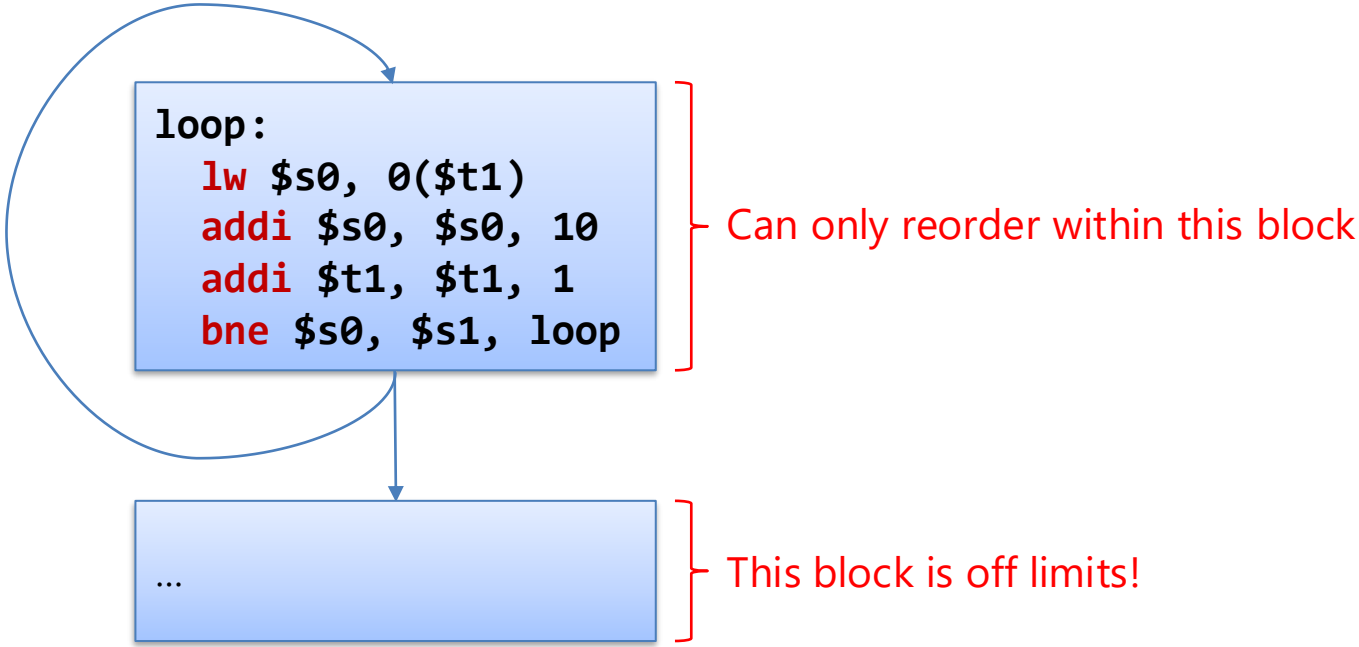
  - Predicates can convert if-then-else code into one big block

```
lw $t1, 0($s0)
pne $p1, $s1, $s2
lw $t2, 1($s0), !$p1
addi $t1, $t1, 1
lw $t2, 2($s0), $p1
```

Can reorder within a larger window!

But Predicates Cannot Remove Loopback Branches

- **Loops** are particularly challenging to the compiler. Why?
  - Scheduling is limited to within the loop
  - For tight loops, not much compiler can do with a handful of insts



Let's use this loop as a running example

```
for (i = 0; i < 100; i++)
  a[i] = a[i] + x;
```

Is equivalent to

```
for (p = &a[0]; p < &a[100]; p++)
  *p = *p + x;
```

Translated to MIPS

```
// Map p → $s1, x → $s2, and &a[100] → $s3
Loop:
  lw   $t0, 0($s1)          // $t0 = *p
  add  $t0, $t0, $s2        // $t0 = $t0 + x
  sw   $t0, 0($s1)          // *p = $t0
  addi $s1, $s1, 4          // p++
  blt  $s1, $s3, Loop       // loopback if p < &a[100]
```

This is the loop schedule on VLIW as-is

- This is the best the compiler can do with current order:

```
Loop:
  lw   $t0, 0($s1)
  add  $t0, $t0, $s2
  sw   $t0, 0($s1)
  addi $s1, $s1, 4
  blt  $s1, $s3, Loop
```

	ALU/Branch Op	Load/Store Op
Loop:	nop	lw $t0, 0($s1)
	nop	nop
	add $t0, $t0, $s2	nop
	addi $s1, $s1, 4	sw $t0, 0($s1)
	blt $s1, $s3, Loop	nop

- **lw** and **add** separated by 2 cycles → due to use-after-load on **$t0**
- **add** and **sw** separated by 1 cycle → due to dependence on **$t0**
- **addi** and **sw** on same cycle → **sw** uses old value of **$s1**
- **IPC = 5 / 5 = 1**. On a 2-wide VLIW. We need to better!
  - But chain of dependencies prevents any reordering… or does it?

Dependencies can be broken (with proper patch-up)

- We broke the WAR (Write-After-Read) dependence on **$s1!**

```
Loop:
  lw   $t0, 0($s1)
  addi $s1, $s1, 4
  add  $t0, $t0, $s2
  sw   $t0, -4($s1)
  blt  $s1, $s3, Loop
```

	ALU/Branch Op	Load/Store Op
Loop:	addi $s1, $s1, 4	lw $t0, 0($s1)
	nop	nop
	add $t0, $t0, $s2	nop
	blt $s1, $s3, Loop	sw $t0, -4($s1)

- Now, **sw** uses the new value of **$s1** produced by **addi**
- The compiler compensates by changing the **sw** offset by -4:
  - **sw $t0, 0($s1)** → **sw $t0, -4($s1)**
- Now, **IPC = 5 / 4 = 1.25**. Can we do even better?

Expand scheduling window using *multiple iterations* of the loop!

Control Dependency Removal: Loop Unrolling

What is Loop Unrolling?

- **Loop unrolling** : a compiler technique to enlarge loop body
  - By duplicating loop body for an X number of iterations

```
for(i = 0; i < 100; i++)
  a[i] = a[i] + x;
```

Original loop

Unrolled loop (2X)

```
for(i = 0; i < 100; i += 2){
  a[i] = a[i] + x;
  a[i+1] = a[i+1] + x;
}
```

- What does this buy us?
  - And **expanded scheduling window** to reorder and hide bubbles
  - And less instructions to execute as a whole
    - Less frequent loop branches
    - Two i++ are merged into one i+= 2

1. Unroll the loop

```
Loop:
  lw   $t0, 0($s1)
  addi $s1, $s1, 4
  add  $t0, $t0, $s2
  sw   $t0, -4($s1)
  blt  $s1, $s3, Loop
```

Unroll 2X

```
Loop:
  lw   $t0, 0($s1)
  addi $s1, $s1, 4
  add  $t0, $t0, $s2
  sw   $t0, -4($s1)

  lw   $t1, 0($s1)
  addi $s1, $s1, 4
  add  $t1, $t1, $s2
  sw   $t1, -4($s1)

  blt  $s1, $s3, Loop
```

- Instructions are duplicated but using **`$t1`** instead of **`$t0`**
- This is intentional to minimize false dependencies during reordering

2. Interleave iterations to space out dependencies

```
Loop:
  lw   $t0, 0($s1)
  lw   $t1, 4($s1)

  addi $s1, $s1, 4
  addi $s1, $s1, 4

  add  $t0, $t0, $s2
  add  $t1, $t1, $s2

  sw   $t0, -8($s1)
  sw   $t1, -4($s1)

  blt  $s1, $s3, Loop
```

Reorder!

```
Loop:
  lw   $t0, 0($s1)
  addi $s1, $s1, 4
  add  $t0, $t0, $s2
  sw   $t0, -4($s1)

  lw   $t1, 0($s1)
  addi $s1, $s1, 4
  add  $t1, $t1, $s2
  sw   $t1, -4($s1)

  blt  $s1, $s3, Loop
```

- Interleaving iterations spaces out dependencies by 2X (unroll factor)

3. Merge induction variable increment

```
Loop:
  lw   $t0, 0($s1)
  lw   $t1, 4($s1)

  addi $s1, $s1, 4
  addi $s1, $s1, 4

  add  $t0, $t0, $s2
  add  $t1, $t1, $s2

  sw   $t0, -8($s1)
  sw   $t1, -4($s1)

  blt  $s1, $s3, Loop
```

Merge

```
Loop:
  lw   $t0, 0($s1)
  lw   $t1, 4($s1)

  addi $s1, $s1, 8

  add  $t0, $t0, $s2
  add  $t1, $t1, $s2

  sw   $t0, -8($s1)
  sw   $t1, -4($s1)

  blt  $s1, $s3, Loop
```

- Two `addi $s1, $s1, 4` are merged into `addi $s1, $s1, 8`

4. Schedule unrolled loop onto VLIW

```
Loop:
  lw   $t0, 0($s1)
  lw   $t1, 4($s1)
  addi $s1, $s1, 8
  add  $t0, $t0, $s2
  add  $t1, $t1, $s2
  sw   $t0, -8($s1)
  sw   $t1, -4($s1)
  blt  $s1, $s3, Loop
```

	ALU/Branch Op	Load/Store Op
Loop:	addi $s1, $s1, 8	lw $t0, 0($s1)
	nop	lw $t1, 4($s1)
	add $t0, $t0, $s2	nop
	add $t1, $t1, $s2	sw $t0, -8($s1)
	blt $s1, $s3, Loop	sw $t1, -4($s1)

- Now we spend 5 cycles for 2 iterations of the loop
  - So, 5 / 2 = **2.5 cycles per iteration**
  - Much better than the previous 4 cycles for 1 iteration!

Unrolling loop 4X buys us even more speedup!

- 4X Unrolled loop converted to VLIW:

	ALU/Branch Op	Load/Store Op	Inst
Loop:	addi $s1, $s1, 16	lw $t0, 0($s1)	1
	nop	lw $t1, -12($s1)	2
	add $t0, $t0, $s2	lw $t2, -8($s1)	3
	add $t1, $t1, $s2	lw $t3, -4($s1)	4
	add $t2, $t2, $s2	sw $t0, -16($s1)	5
	add $t3, $t3, $s2	sw $t1, -12($s1)	6
	nop	sw $t2, -8($s1)	7
	blt $s1, $s3, Loop	sw $t3, -4($s1)	8

- Now we spend 8 cycles for 4 iterations of the loop
  - So, 8 / 4 = **2 cycles per iteration**
  - Even better than 2.5 cycles per iteration for 2X unrolling

Unrolling beyond that won't buys us anything

- 8X Unrolled loop converted to VLIW (16/8 = **2 cycles per iteration**) :

	ALU/Branch Op	Load/Store Op	Inst
Loop:	addi $s1, $s1, 32	lw $t0, 0($s1)	1
	nop	lw $t1, -28($s1)	2
	add $t0, $t0, $s2	lw $t2, -24($s1)	3
	add $t1, $t1, $s2	lw $t3, -20($s1)	4
	add $t2, $t2, $s2	lw $t4, -16($s1)	5
	add $t3, $t3, $s2	lw $t5, -12($s1)	6
	add $t4, $t4, $s2	lw $t6, -8($s1)	7
	add $t5, $t5, $s2	lw $t7, -4($s1)	8
	add $t6, $t6, $s2	sw $t0, -32($s1)	9
	add $t7, $t7, $s2	sw $t1, -28($s1)	10
	nop	sw $t2, -24($s1)	11
	nop	sw $t3, -20($s1)	12
	nop	sw $t4, -16($s1)	13
	nop	sw $t5, -12($s1)	14
	nop	sw $t6, -8($s1)	15
	blt $s1, $s3, Loop	sw $t7, -4($s1)	16

When should the compiler stop unrolling?

- When dependencies are already sufficiently spaced far apart.

- But constraints can prevent unrolling even before that:
  1. Limitation in number of registers
     - More unrolling uses more registers $t0, $t1, $t2, …
     - For this reason, VLIW ISAs have many more registers than MIPS – Intel Itanium has 256 registers!
  2. Limitation in code space
     - More unrolling means more code bloat
     - Embedded processors don't have lots of code memory
     - Even general-purpose processors have limited instruction cache (Code that overflows i-cache can lead to lots of cache misses)

Data Dependency Removal: Static Single Assignment

Next up: Data Dependencies

- Removing control dependencies expanded **instruction window**
  - Instruction window: block of code without branches
  - How? Through predication and loop unrolling.

- Large instruction window gives compiler freedom to schedule

- Now what still constrains compiler scheduling?
  - Data dependencies!

- Not all data dependencies are born equal
  - Some dependencies can be elided by smart code generation! (By adopting something called Static Single Assignment)

Three Types of Data Dependencies

- The 3 types of data dependencies:
  - **RAW (Read-After-Write)**
    - True dependency where data flows from producer to consumer
    - If reordered, data flow is broken and read loads wrong value
  - **WAR (Write-After-Read)**
    - Write to a location that has just been read
    - If reordered, creates a data flow that did not exist
  - **WAW (Write-After-Write)**:
    - Write to a location that has just been written
    - If reordered, location ends up with a stale value

- How about Read-After-Read? Not a data dependency.

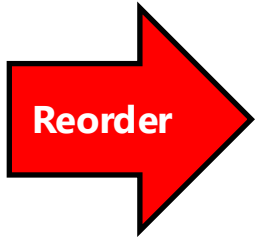
Read-After-Write (RAW) Dependency

- RAW dependencies are also called ***true dependencies***
  - "true" because RAW is the only dependence with actual data flow

- Suppose we reorder this snippet of code:

RAW!

```
lw t0, 0(s0)
addi t1, t0, 4
```




```
addi t1, t0, 4
lw t0, 0(s0)
```

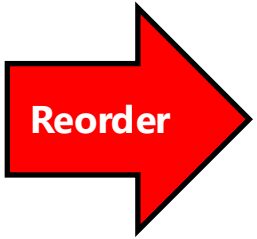
- The code is incorrect because now **`t1`** has a wrong value
  - Value in **`t0`** can be read only after being written to
  - No amount of compiler tinkering will allow this reordering

Write-After-Read (WAR) Dependency

- WAR dependencies are also called ***anti-dependencies***
  - In the sense that they are the opposite of true dependencies

- Suppose we reorder this snippet of code:

```
      lw t0, 0(s0)
      addi t1, t0, 4
WAR!  lw t0, 0(s1)
```




```
lw t0, 0(s0)
lw t0, 0(s1)
addi t1, t0, 4
```

- The code is again incorrect because **`t1`** has the wrong value
  - **`addi`** should not read **`t0`** produced by **`lw t0, 0(s1)`**
- Q: Is there a way for **`addi`** to *not use* that value?
  - **`t0`** and **`t0`** contain different values. Why re-use the same register?
  - Just **rename** register **`t0`** to some other register!

Removing WAR with SSA

- ***Static Single Assignment***:
  - Renaming registers when a different value is stored into it
  - A register is assigned a value only a single time (never reused)

- Reordering after converting to SSA form:

```
lw t0, 0(s0)
addi t1, t0, 4
lw t2, 0(s1)
```

NO WAR!

Reorder

```
lw t0, 0(s0)
lw t2, 0(s1)
addi t1, t0, 4
```

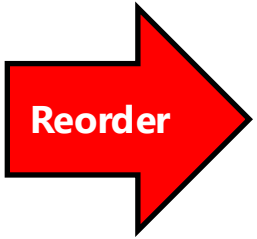
- Note how destination registers always use a new register
  - Yes, if you do this, you will need lots of registers
  - But, no more WAR dependencies!

Write-After-Write (WAW) Dependency

- WAW dependencies are also called ***false dependencies***
  - In the sense that they are not real dependencies

- Suppose we reorder this snippet of code:

```
WAW!  lw t0, 0(s0)
      lw t0, 0(s1)
      addi t1, t0, 4
```




```
lw t0, 0(s1)
lw t0, 0(s0)
addi t1, t0, 4
```

- The code is again incorrect because **`t1`** has the wrong value
  - **`addi`** should not read **`t0`** produced by **`lw t0, 0(s0)`**
- Q: Is there a way for **`addi`** to *not use* that value?
  - Again, **rename** register **`t0`** to some other register!

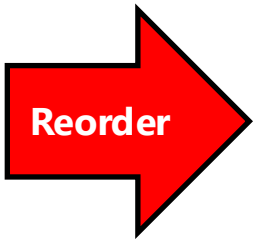
Removing WAW with SSA

- Again, Static Single Assignment (SSA) to the rescue!
  - SSA removes both WAR and WAW dependencies

- Reordering after converting to SSA form:

NO WAW!

```
lw t0, 0(s0)
lw t1, 0(s1)
addi t2, t1, 4
```




```
lw t1, 0(s1)
lw t0, 0(s0)
addi t2, t1, 4
```

- SSA form is now the norm in all mature compilers
  - Clang / LLVM ("Apple" Compiler)
  - GCC (GNU C Compiler)
  - Java Hotspot / OpenJDK Compiler
  - Chrome JavaScript Compiler

Final Step:
List Scheduling

How does the compiler schedule instructions?

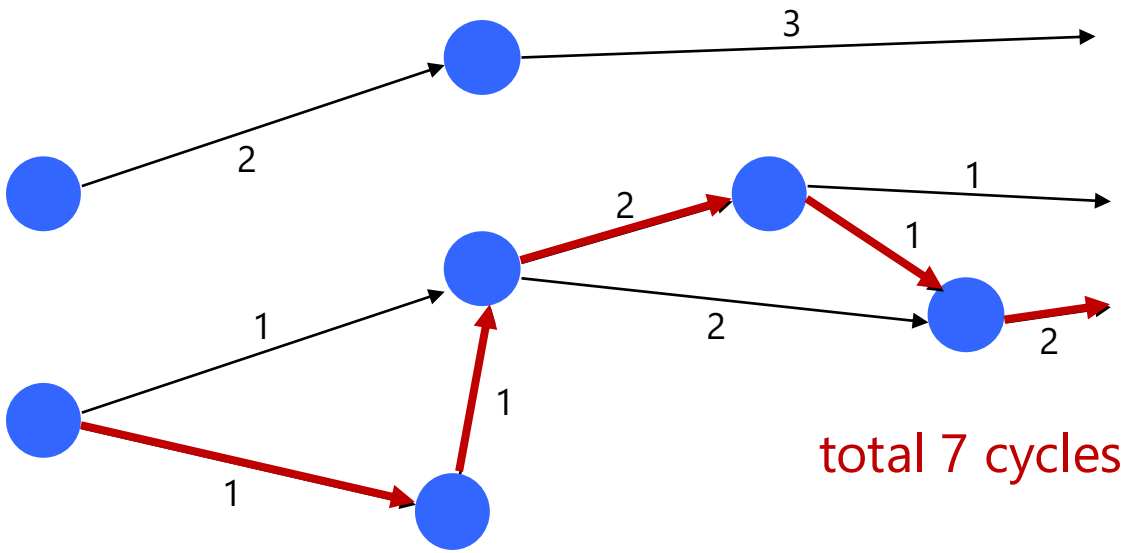
- After control and data dependencies are removed, time to schedule!

- A popular scheduling algorithm is ***list scheduling***
  - Principle: list instructions in some order of priority and schedule
  - Popular heuristic: Prioritize **critical path** instructions

- List scheduling can be used with any statically scheduled processor
  - Simple single-issue statically scheduled processor (not just VLIW)
  - GPUs are also statically scheduled using list scheduling

Critical Path in Code

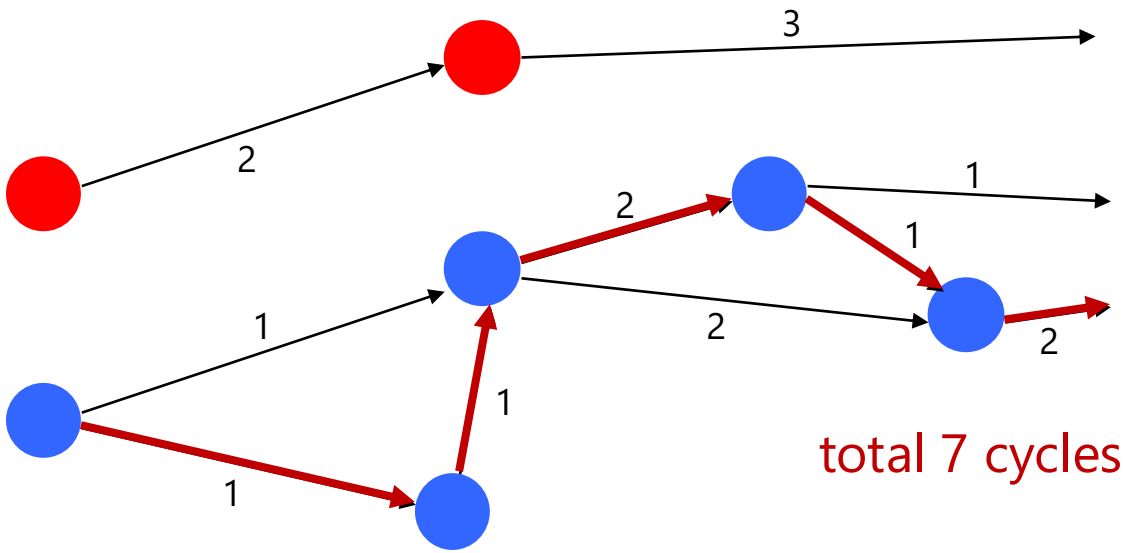
- At below is a data dependence graph for a code with 7 instructions
  - Nodes are instructions
  - Arrows are data dependencies annotated with required delay
- Q: How long is the critical path in this code?



That means, at minimum, this code will take 7 cycles, period. Regardless of how wide your processor is or how well you do your scheduling.

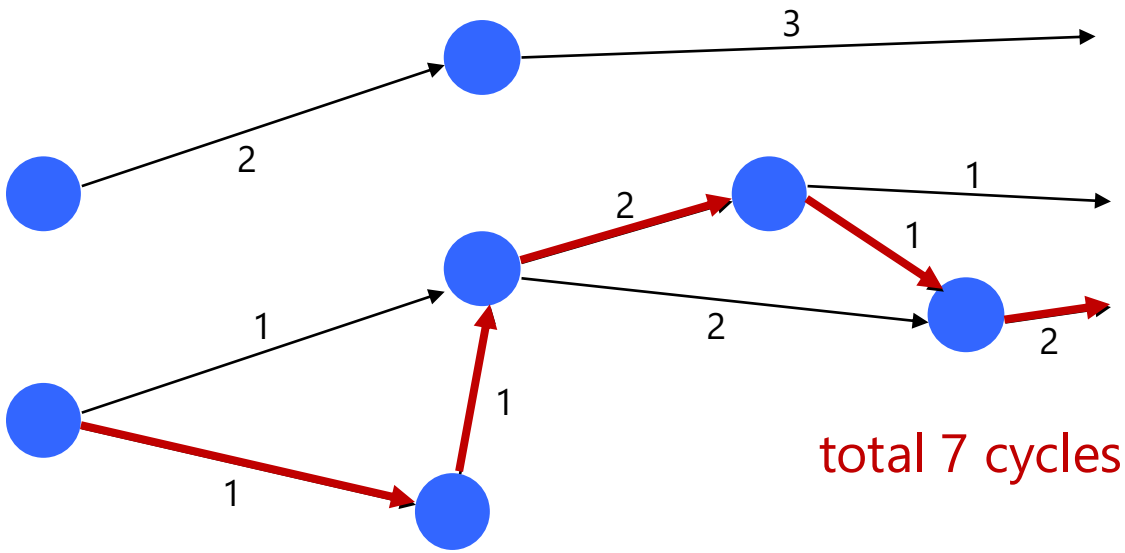
Instruction Level Parallelism (ILP)

- The 7 cycles is achievable thanks to **instruction level parallelism**
  - Property of graph that allows parallel execution of instructions
  - The nodes marked in red can execute in parallel with blue nodes
- This tell us that this code is where a VLIW processor can shine



Prioritizing Critical Path is Usually a Good Idea

- The critical path length is 7 cycles but that is not always achievable
  - If CPU is not wide enough for the available parallelism
  - Or if CPU is wide enough but compiler does bad scheduling
- When critical path instruction is ready, good idea to not delay it!



Q: Can the graph be cyclic?

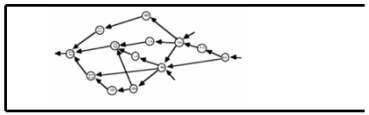
List Scheduling is a Greedy Algorithm

- Steps:
  1. Create a **data dependence graph**
  2. Assign a **priority** to each node (instruction)
     - Priority = **critical path** length starting from that node
  3. Schedule nodes one by one starting from **ready** instructions
     - Ready = all dependencies have been fulfilled
       (Initially, only roots of dependency chains are ready)
     - When there are multiple nodes that are ready
       → Greedily choose the node with the highest priority
- It is a greedy algorithm because choices can be suboptimal
  - Critical Path (CP) calculation was based on unlimited resources
  - CPUs have limited resources that require adjustments to CP
    - Limited number of FUs, limited number of registers

List Scheduling Example

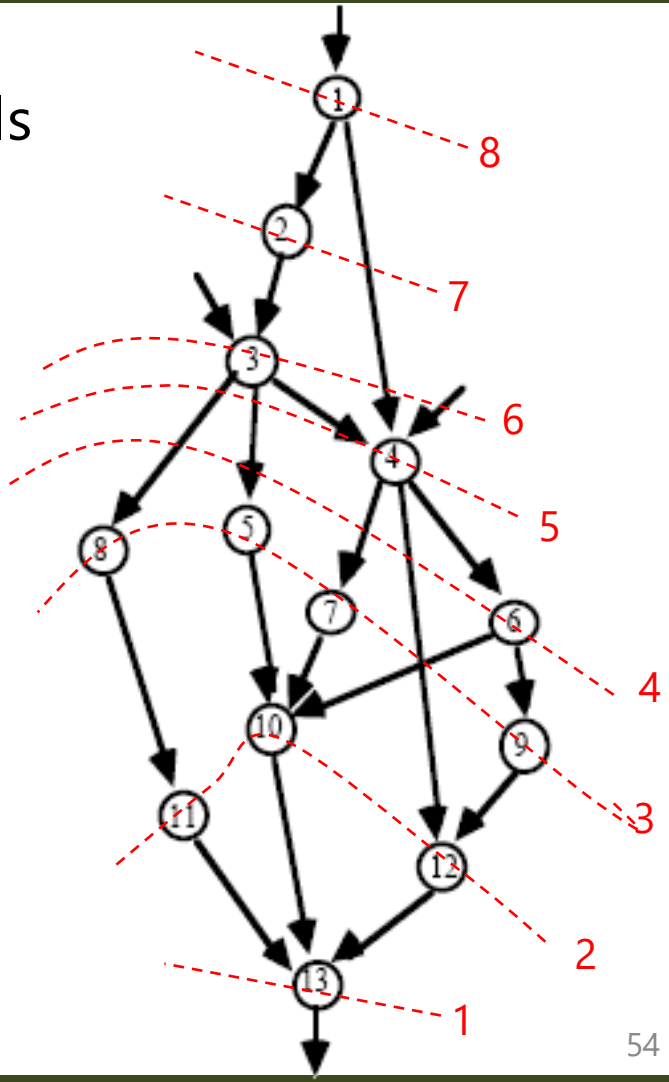
- Assume all edges have a delay of 1
  - Red dashed lines indicate priority levels

n insertions +
n removals.
If using priority heap
= O(nlogn)



Ready instructions List

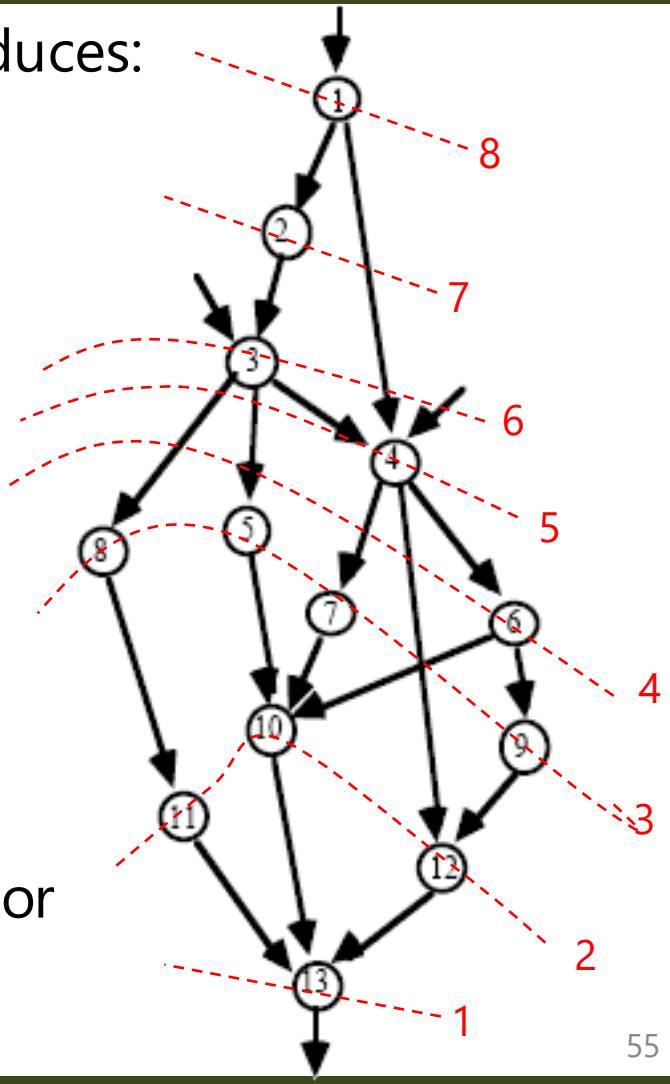
Operation 1	Operation 2



List Scheduling Example

- This is one possible schedule that it produces:

Op 1	Op 2	Ready
1		1
2		2
3		3
4	8	4,5,8
6	7	5,6,7,11
5	9	5,9,11
10	11	10,11,12
12		12
13		13



- 9 cycles. We couldn't achieve 8 cycles!
  - But could've if we had a wider processor

List Scheduling is not optimal

- Optimal scheduling is an **NP-Complete** problem:
  - Ref: *John L. Hennessy and Thomas Gross. "Postpass Code Optimization of Pipeline Constraints", ACM Trans. Program. Lang. Syst., July 1983.*
- List scheduling achieves worst-case **2 – 1/n** of optimal schedule
  - Where **n** is the width of the processor
  - Ref: *R. L. Graham, "Bounds for certain multiprocessing anomalies," in The Bell System Technical Journal, 1966.*

- More elaborate algorithms exist to approach optimal
  - Using constraint solvers or machine learning

- Still, list scheduling using critical path lengths is most widely used

- GCC uses other heuristics in addition to critical path length:
  - https://github.com/gcc-mirror/gcc/blob/master/gcc/haifa-sched.cc